

A research lab you can explain



Follow the direction; each stage has a defined input and output.

The result to remember

01 Three full fine-tuning runs on compatible flows

02 Mean test accuracy: 86.65%

03 Paper result: 97.50% — not reproduced

Packet, header and payload

- 01** Packet: a small unit sent across a network
- 02** Header: delivery and protocol information
- 03** Payload: carried content, possibly encrypted

A flow needs a grouping rule

Conversation A



Flow A

Related packets

Conversation B



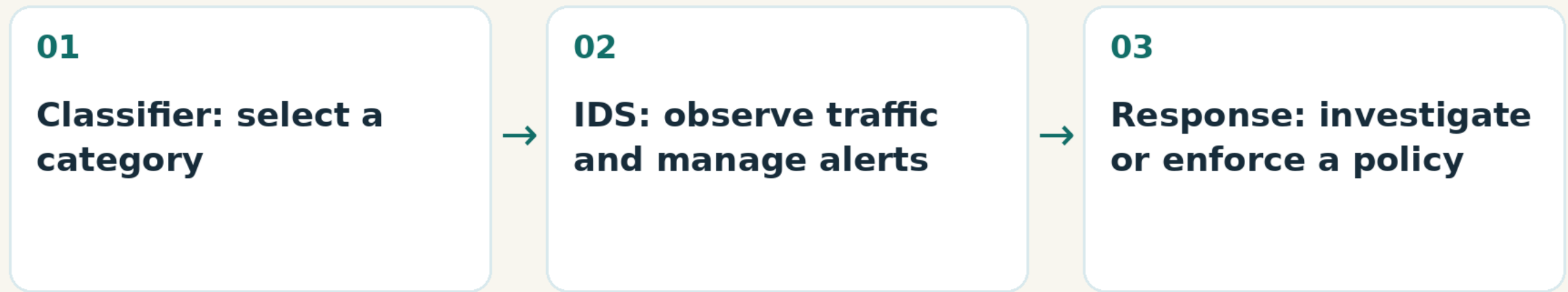
Flow B

Related packets

Illustration: preserve membership and order; proximity in a CSV is insufficient.

Evidence: csv · native · contract | [Links and full text in the transcript](#)

Classifier, detector and response



Follow the direction; each stage has a defined input and output.

How to read an evidence claim

- 01** Implemented: code exists
- 02** Executed: a recorded command ran
- 03** Validated: a named check passed

Your presentation has an evidence trail

- ? Watch this course to learn the story
- ? Use the transcript to explain it again
- ? Use the file guide to find each underlying record

Your turn · 10 seconds

01 Does a working classifier prove a complete live IDS?

Speak or write your answer. The lesson resumes automatically.

Evidence: walkthrough · answers | Links and full text in the transcript

What the attached PDF is

- 01** arXiv 2601.21792v1 · January 2026
- 02** Traffic classification research
- 03** A framework with more than one model variant

Three problems motivate the paper

- 01** Computation cost of long sequences
- 02** Information lost or biased during extraction
- 03** Rare categories hidden by class imbalance

What multimodal means here

01

**Bytes: content
patterns**

02

**Sizes: transmission
shape**

03

**Intervals:
transmission rhythm**

Three parallel feature views from the same flow.

Mamba and the alternative backbone

- 01** Mamba: input-dependent state updates
- 02** NetTrans: a separate attention-based alternative
- 03** Our run uses the Mamba classifier

Read the paper in this order

01 Framework and representation: sections IV-V

02 Model and learning details: section VI

03 System and measurements: sections VII-VIII

A table is not our result

Paper Table IV

97.50%

Our three-seed mean

86.65%

Different runtime and training settings. The cause of the gap remains unresolved.

Two details that prevent overclaiming

?

Paper byte sequence: 401 tokens

?

Our multimodal sequence: 443 tokens

?

Paper online throughput is a separate NetMamba prototype

Your turn · 10 seconds

01 Name one paper claim that has not been reproduced here.

Speak or write your answer. The lesson resumes automatically.

A CSV is a table, not a running database

- 01** Rows: individual records
- 02** Columns: named fields
- 03** This file: 1,410,255 rows and 1,505 columns

One row contains five kinds of metadata

Field group	Recorded meaning
payload_byte_1 ... 1500	Payload positions; numbered consecutively
ttl	Time-to-live field
total_len	Recorded length
protocol / t_delta	Protocol / time-difference field
label	Dataset category

payload_byte_1 ... payload_byte_1500 · ttl / total_len / protocol / t_delta · label: the recorded category

Benign and attack labels are imbalanced

Recorded label	Rows
BENIGN	362,108
DoS Hulk	250,000
DDoS	241,405
DoS GoldenEye	128,122
DoS slowloris	121,097

BENIGN: 362,108 rows · DoS Hulk: 250,000 rows · DDoS: 241,405 rows

Why the small classes matter

Recorded label	Rows
SSH-Patator	48,165
FTP-Patator	31,843
Bot	2,543
PortScan	830
Web Attack – Sql Injection	12

PortScan: 830 rows · Web Attack – Sql Injection: 12 rows · A good overall score can hide rare-class failures

Evidence: csv | Links and full text in the transcript

Odd values are questions to investigate

- 01** Some `t_delta` values are negative
- 02** Minimum observed: `-0.000018`
- 03** Some `total_len` values exceed 1,500

What the profiling actually checked

- 01** Full-file row counts and metadata aggregates
- 02** Label and protocol distributions
- 03** Payload values were not exhaustively validated

The missing information changes the task

- ? No established addresses, ports or flow ID
- ? No established order within each flow
- ? Payload-only rows cannot establish header semantics

Your turn · 10 seconds

01 Why is matching a tensor shape insufficient to establish compatible flow data?

Speak or write your answer. The lesson resumes automatically.

The UNSW extract has the same broad schema

01 79,881 rows

02 1,500 payload columns + 5 metadata fields

03 10 label names

Read the UNSW label distribution

Recorded label	Rows
normal	21,000
generic	17,580
exploits	13,992
fuzzers	12,722
reconnaissance	7,562

normal: 21,000 · generic: 17,580 · exploits: 13,992

Rare labels and protocol variety

Recorded label	Rows
dos	3,397
backdoor	1,239
analysis	1,208
shellcode	1,088
worms	93

dos: 3,397 · backdoor: 1,239 · analysis: 1,208 · shellcode: 1,088 · worms: 93

Related subject, different learning problems

Input collection	Unit / category count
Uploaded CICIDS2017	Packet records / 15 labels
Uploaded UNSW	Packet records / 10 labels
Authors' CICIoT2022 release	Native flows / 6 classes

CICIDS2017 CSV: 15 packet labels · UNSW CSV: 10 packet labels · CICIoT2022 flows: 6 native classes

Why the actual experiment used other data

- 01** Train: 8,323 flows · Validation: 1,040 flows
- 02** Test: 1,041 flows · 6 native classes
- 03** Record the departure from the uploaded CSVs

Compatible data still has limitations

- 01** 5 identical stored inputs across train / validation
- 02** 6 across train / test
- 03** Capture independence and pretraining exposure remain unknown

The defensible connection between the files

- ? The paper defines a flow representation
- ? The CSVs reveal an input-contract mismatch
- ? The repository tests a compatible native route

Your turn · 10 seconds

01 Which data was actually used for training, and why were the uploaded CSVs not used?

Speak or write your answer. The lesson resumes automatically.

Evidence: comparison · csv · native | [Links and full text in the transcript](#)

The actual input is a list of flow objects

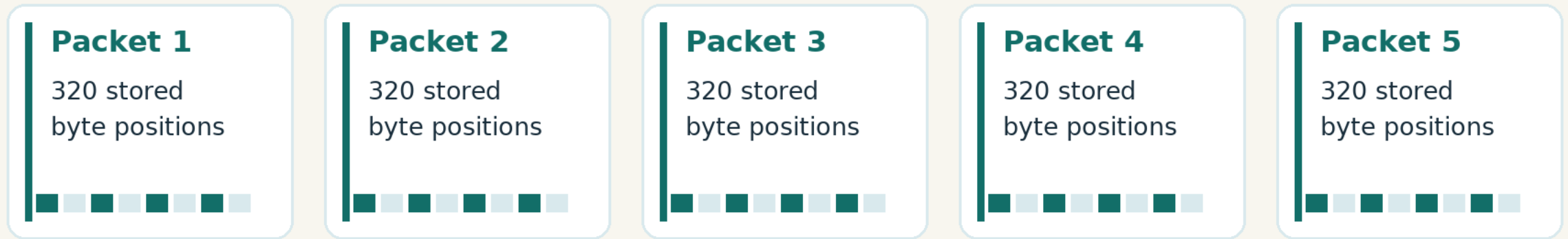
```
[ {  
  "data": ["69 0 0 64", "69 0 0 52"],  
  "sizes": "64 52",  
  "intervals": "0 0.001",  
  "label": 0, "name": "6-Attacks-1-Flood"  
} ]
```

data: packet byte strings

sizes and intervals: numeric strings

label and name: known training category

Bytes: five packets, 320 stored bytes each



$$5 \times 320 = 1,600 \text{ stored byte positions}$$

Paper: 80 header + 240 payload allocation. Released extraction history is unverified.

Normalization changes the numerical scale

```
normalized_byte = (byte / 255 - 0.5) / 0.5
```

```
0 → -1.0
```

```
255 → +1.0
```

```
byte tensor: [batch, 1, 1, 1600]
```

byte / 255 → range 0 to 1

Subtract 0.5, divide by 0.5 →
range -1 to 1

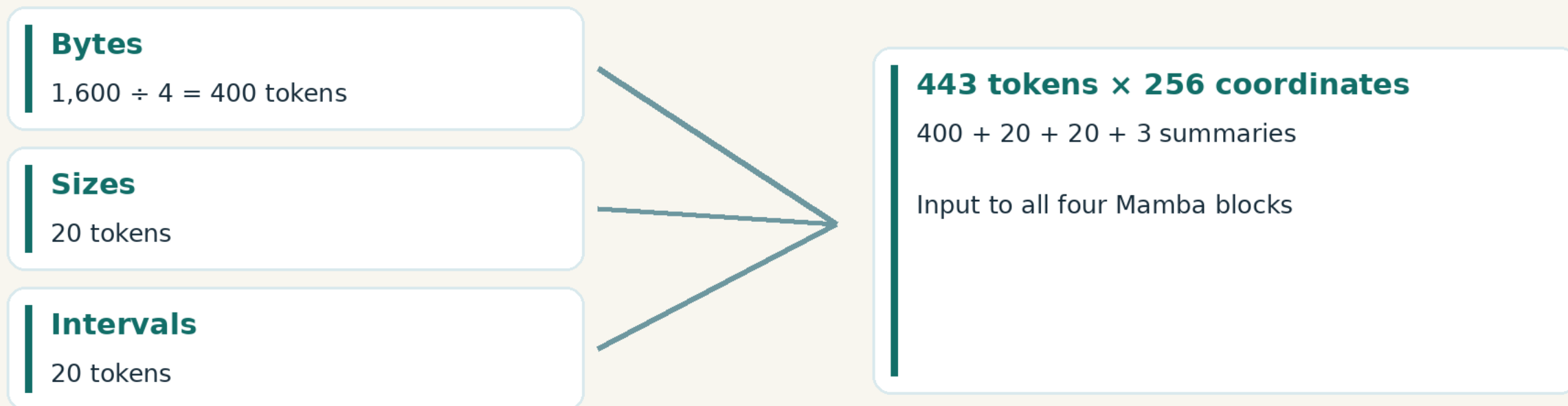
Tensor shape: batch × 1 × 1 ×
1,600

Sizes and intervals use their own rules

Example input	Transformed value
Observed gap $x = 0$	0.5
Observed gap $x = 1$	$2 / 3$
Padded interval	1
Size larger than 1,500	Clipped to 1,500

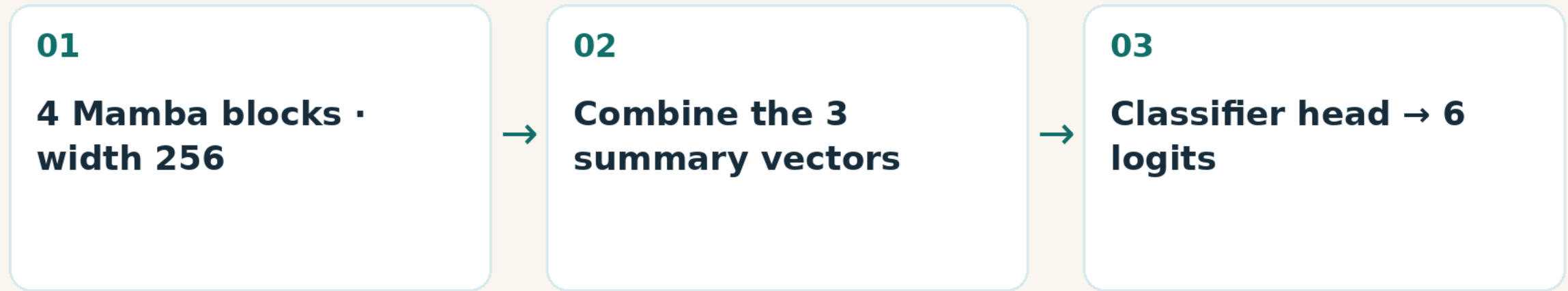
First 20 sizes: clipped to 0 ... 1,500 · First 20 gaps: $(1 + x) / (2 + x)$ · Missing values use the loader's padding rules

The 443-token calculation



Tokens are numerical input units; they are not language-model words.

Four blocks produce six raw outputs



Complete classifier: 1,870,080 parameters.

Explain the input contract back

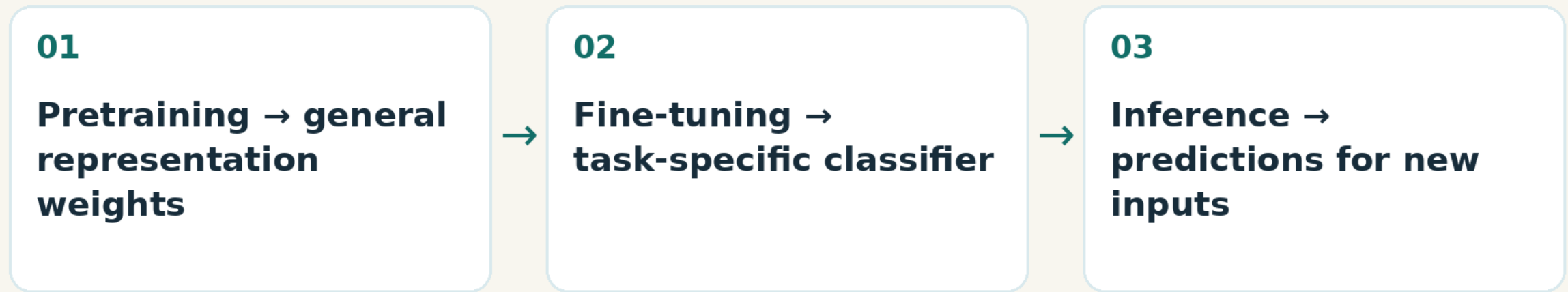
- ? Meaning and origin must match
- ? Dimensions and transforms must match
- ? Class meanings must match the checkpoint

Your turn · 10 seconds

01 Describe the three feature views and the six-number output.

Speak or write your answer. The lesson resumes automatically.

Three activities, three different outputs



Follow the direction; each stage has a defined input and output.

Pretraining hides part of the input

01 Mask byte content and selected feature values

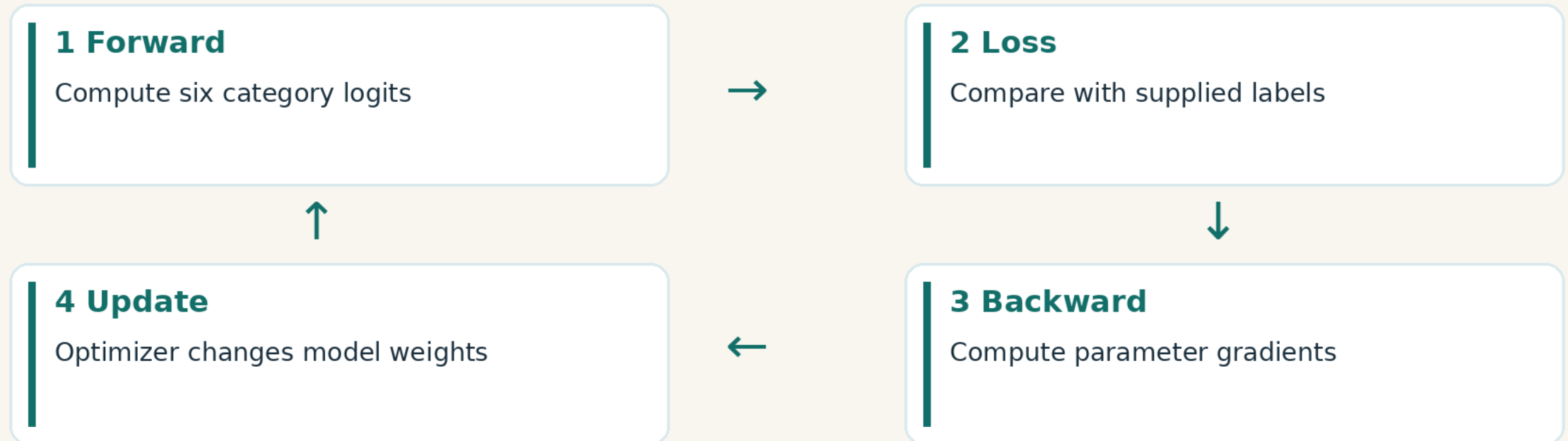
02 Reconstruct the missing information

03 Compare reconstruction with the original values

What our pretraining check actually did

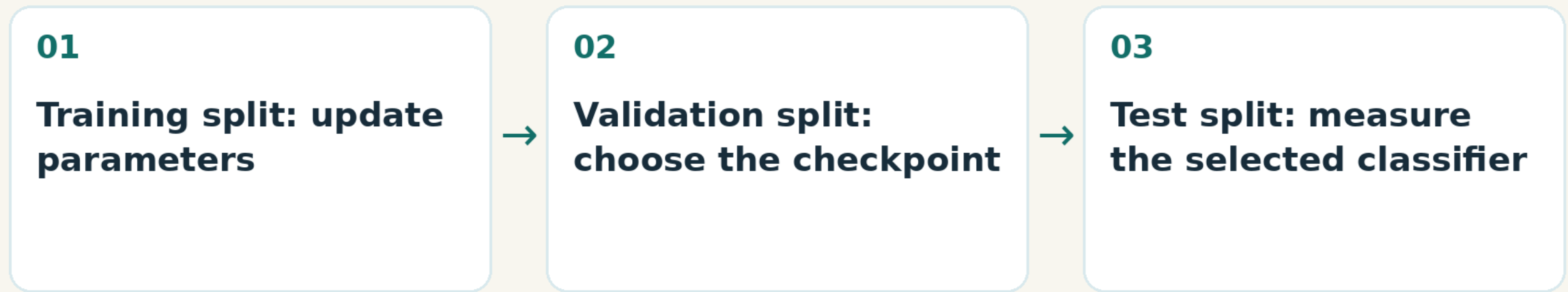
- 01** 2 completed epochs · 132 updates
- 02** Loss: 9.51953 → 6.12702
- 03** Separate from the checkpoint used for full fine-tuning

A labeled batch changes the classifier



Conceptual training cycle · repeated across batches and epochs

Three splits prevent one obvious shortcut



Follow the direction; each stage has a defined input and output.

The completed training protocol

- 01** Seeds 0, 1 and 2
- 02** 120 epochs and 7,920 updates per run
- 03** Batch 128; effective learning rate 0.001

A successful reload has a narrow meaning

- ? Strict loading checks all expected tensors
- ? Model state and class meanings must stay together
- ? New accuracy requires known labels

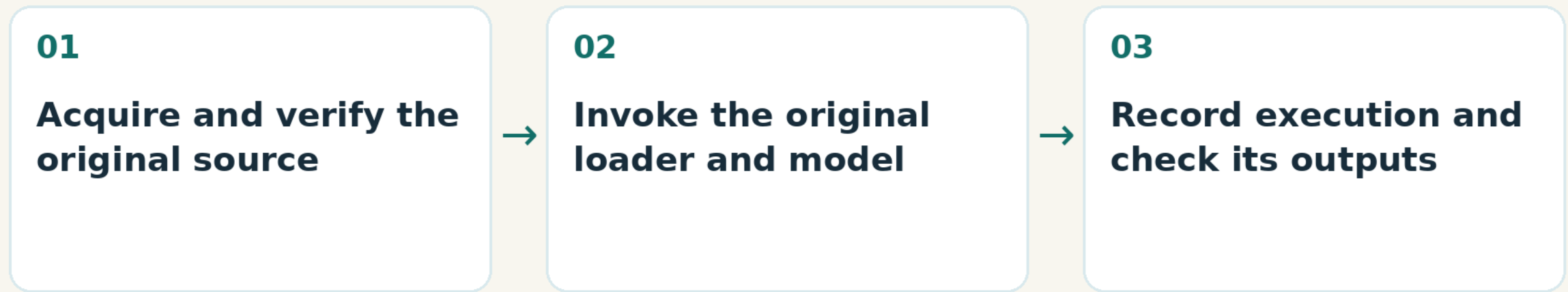
Your turn · 10 seconds

01 What does each of pretraining, fine-tuning and inference produce?

Speak or write your answer. The lesson resumes automatically.

Evidence: transfer · contract · runbook | Links and full text in the transcript

Our code surrounds the authors' code



Follow the direction; each stage has a defined input and output.

repro.py is the experiment coordinator

```
python repro.py fetch
python repro.py validate --data assets/data/ciciot2022

python repro.py finetune ...
python repro.py pretrain ...
```

fetch and validate

pretrain and finetune

Fresh output directories and run manifests

evaluate.py answers: how accurate is this model?

- 01** Labeled native test flows
- 02** Strict saved-classifier loading
- 03** Native evaluation metrics

predict.py answers: which category fits this flow?

- 01** Unlabeled native flow input
- 02** Six logits, scores and a class name
- 03** No accuracy invented without labels

replay.py connects execution to explanation



Follow the direction; each stage has a defined input and output.

Configurations and assets define the recipe

- 01** configs: settings, hashes and class mapping
- 02** assets: downloaded data and weights, kept locally
- 03** upstream/NetMambaPlus: pinned source, kept locally

Read a run in this order

? Manifest: what was requested and what completed

? Log: what execution reported

? Metrics and predictions: what was measured

Your turn · 10 seconds

- 01** Which scripts handle unlabeled prediction and labeled evaluation? Where is run provenance recorded?

Speak or write your answer. The lesson resumes automatically.

Three runs, one fixed test set



Recorded test accuracy · same 1,041-flow test set · sample SD 4.00 percentage points

Evidence: results | [Links and full text in the transcript](#)

Mean and spread answer different questions

01 Mean: 86.65%

02 Sample standard deviation: 4.00 percentage points

03 Not a confidence interval or an independent holdout study

Accuracy, precision, recall and F1

- 01** Accuracy: correct predictions / all predictions
- 02** Precision: how often a predicted category is right
- 03** Recall: how much of a true category was found

Read a confusion matrix

Seed 0 · each cell counts labeled test flows

	0	1	2	3	4	5
0 Flood	185	14	0	0	1	0
1 RTSP Brute Force	1	199	0	0	0	0
2 Audio	0	0	185	1	12	2
3 Other	0	1	1	37	0	2
4 Cameras	0	0	7	0	168	25
5 Home Automation	1	1	6	0	16	176

Rows: known category

Columns: predicted category

Off-diagonal cells are mistakes

Rows: known category · Columns: predicted category (same numbers)

Rechecking a result is different from retraining

- 01** Native evaluation, strict reload and replay agree
- 02** Fresh GB10 environment: 1,041 predictions match
- 03** All 6,246 logits also match in that recheck

The paper gap is still an open question

01 Paper: different runtime and training conditions

02 Released checkpoint history is incomplete

03 No experiment has isolated the gap's cause

Choose wording the evidence can support

- ? “Three measured training runs completed.”
- ? “The frozen classifier result was reexecuted.”
- ? “Customer-network accuracy remains unmeasured.”

Your turn · 10 seconds

01 Explain the mean and spread without claiming customer-network validation.

Speak or write your answer. The lesson resumes automatically.

Evidence: results · verification | Links and full text in the transcript

Open the recorded replay

Input · bytes + sizes + timing → NetMamba+ · 443 tokens / flow → Output · six class scores

1,041 / 1,041 flows displayed	91.26% full test accuracy	91.58% full test macro F1	402 displayed attack-class predictions
----------------------------------	------------------------------	------------------------------	---

[Play replay](#) [Reset](#) [Show all](#) Display pace [Talk-through · 600 ms / row](#) Rows [All displayed flows](#)

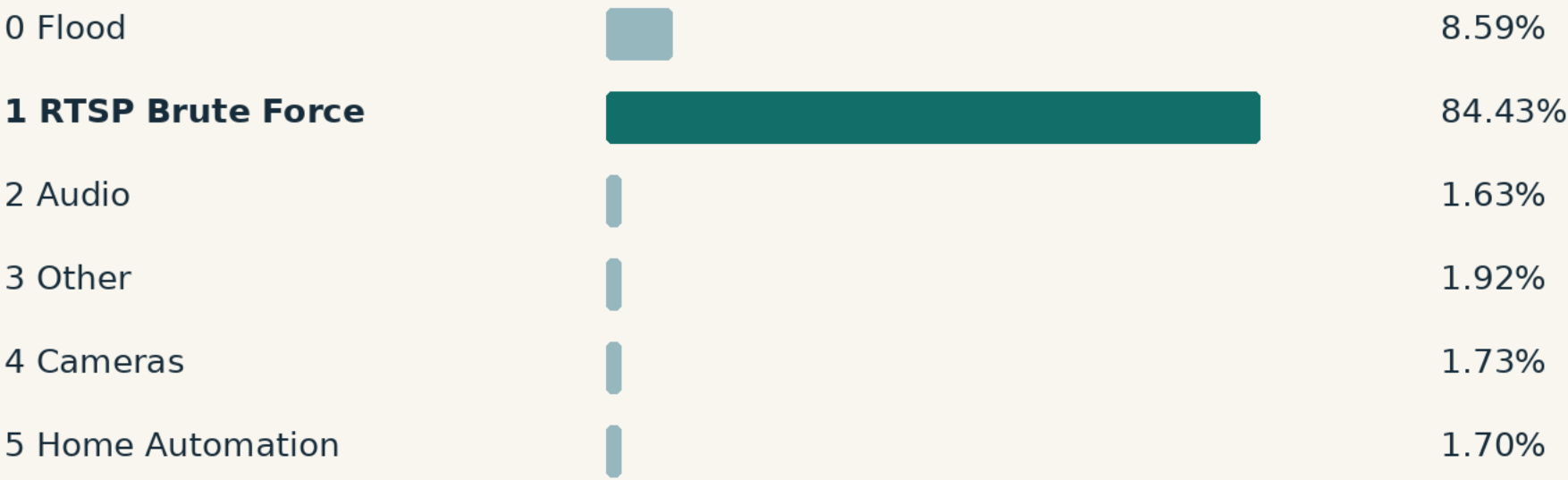
Runs without a GPU or Python · Displays saved seed-0 predictions · Opening it does not run new inference

Evidence: prediction · walkthrough | Links and full text in the transcript

Choose the demo model before inspecting outcomes

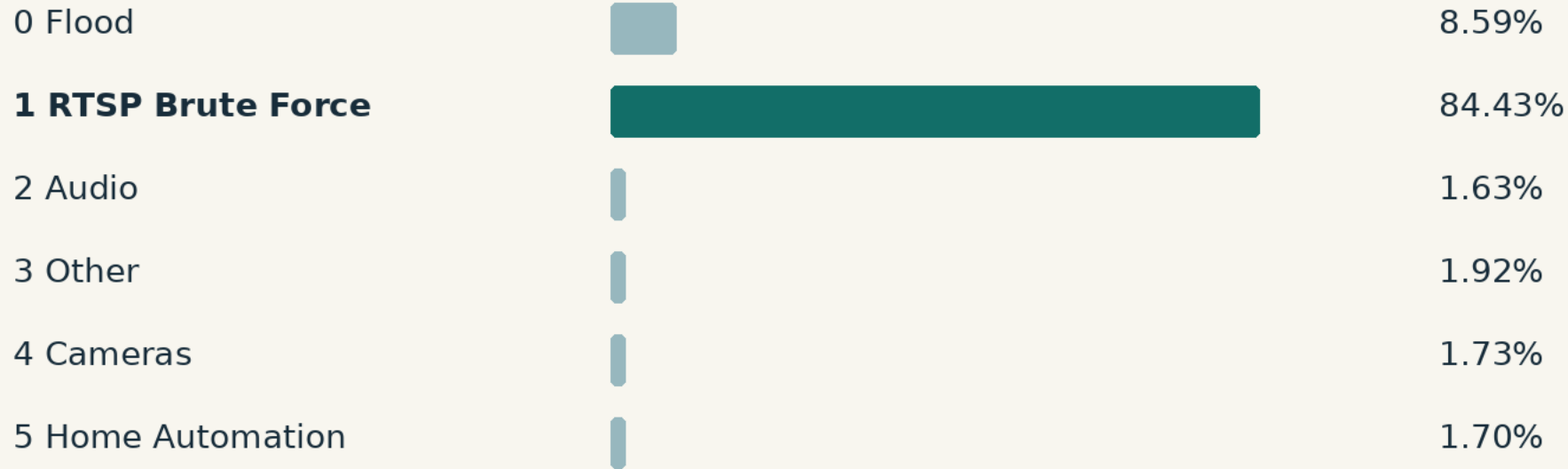
- 01** Seed 0 is the declared demonstration model
- 02** All three seeds remain visible in the results
- 03** A convenient demo is not a new evaluation protocol

Inspect JSON row 635



Known label: reveal next

Reveal the known label



Known label: 3 — Other. Prediction is wrong.

Evidence: [prediction](#) | [Links and full text in the transcript](#)

New inference starts outside the browser

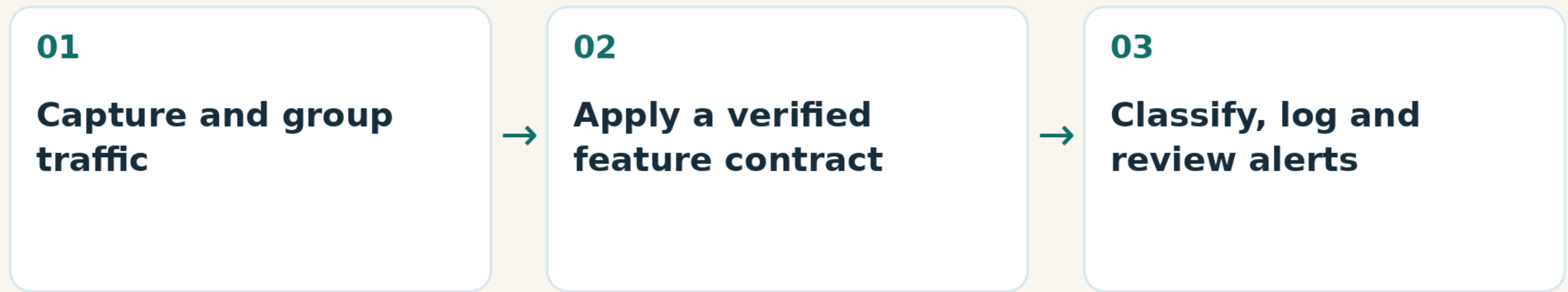
```
python predict.py \  
  --flows path/to/unlabeled-flows.json \  
  --checkpoint runs/ciciot-ft-seed0/checkpoint-best.pth \  
  --output runs/predictions-new
```

Supply already assembled native flows

Load the checked classifier and class mapping

Write predictions to a fresh output directory

A simple next IDS prototype



Follow the direction; each stage has a defined input and output.

Practice the demonstration handoff

- ? Show the replay and one actual error
- ? State what runs locally and what is recorded
- ? Separate a category score from an alert policy

Your turn · 10 seconds

01 Why was the 84.43% display score insufficient to trust this prediction?

Speak or write your answer. The lesson resumes automatically.

Evidence: prediction · answers | Links and full text in the transcript

Choose one of three setup routes

- 01** Present: browser or document viewer
- 02** Recheck: Git and Python
- 03** Execute the model: checked Linux CUDA environment

Clone and run the portable checks

```
git clone https://github.com/buffbeefalo/netmambaplus-reproduction.git
cd netmambaplus-reproduction
python -m unittest discover -s tests -v
python tools/verify_package.py
python tools/verify_course_video.py
python tools/verify_video_course_v3.py
python tools/build_video_page.py --check
```

Clone the current repository

Run the unit suite and package checks

These checks do not download or train the model

What cross-platform CI establishes

- 01** Linux x86-64, Windows x86-64, macOS ARM64
- 02** Python 3.10 and 3.12: six combinations
- 03** Hosted jobs run portable checks, not GPU training

The model environment has native dependencies

- 01** Python 3.12 · Torch 2.9.1 + CUDA 13.0
- 02** Pinned Mamba and causal-convolution extensions
- 03** Compiler, driver and GPU architecture must agree

Pass two gates before a new experiment

```
python tools/check_gpu_runtime.py \  
  --output runs/cuda-ops-new.json  
python tools/check_model_runtime.py \  
  --output runs/cuda-model-new \  
  --build-report runs/cuda-build-new/build-report.json
```

Numerical check: forward and backward comparisons

Complete-model check: update, save, reload, infer

Keep the build report with the runtime results

Know what is still platform-specific

- 01** GB10: measured training and inference
- 02** Other NVIDIA GPUs: build route, machine-specific validation needed
- 03** CPU, Apple MPS, NPU: no validated model backend

Match the test to the claim

- ? Document checks → package integrity
- ? GPU gates → execution correctness within their scope
- ? Labeled holdout → measured predictive performance

Your turn · 10 seconds

01 Which setup is enough to present, and which setup is needed for new model predictions?

Speak or write your answer. The lesson resumes automatically.

Evidence: support · verification | Links and full text in the transcript

Use the file guide as the complete index

- 01** One entry for every tracked file
- 02** Separate explanations for local downloads and run outputs
- 03** Links take you from an explanation to its evidence

Root files, configuration and dependencies

01 README and four entry points: start and execute

02 configs: experiment recipes and identities

03 requirements: packages grouped by activity

Tools: acquire, build, measure and explain

- 01** fetch_assets / profile_csvs / capture_runtime
- 02** build_cuda / build_gb10 / runtime checks
- 03** benchmark / probe_export / review_experiments

Tools: documents, replay and this video

- 01** Package and guide builders produce teaching artifacts
- 02** Replay rendering and recording preserve demonstrations
- 03** Narration, video and page builders share one lesson source

Tests, workflows, documents and evidence

- 01** tests: controlled regression cases
- 02** workflows: portable CI and Pages publication
- 03** docs: explanations, evidence, audits and research records

Choose the right download

- 01** Current course: one MP4, transcript, captions and companion slides
- 02** Historical customer bundle: fixed original experiment package
- 03** Current support matrix: latest tested setup boundaries

A practical reading order

- ? README → course → file guide
- ? Runbook and support matrix before executing commands
- ? Results → manifests → logs when checking a claim

Your turn · 10 seconds

01 Where would you verify a platform claim and an actual training result?

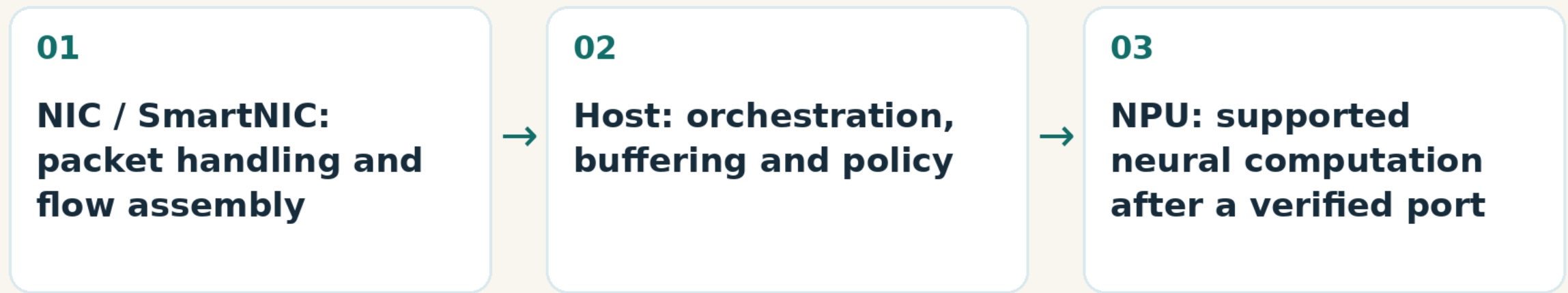
Speak or write your answer. The lesson resumes automatically.

Evidence: walkthrough · support · results | Links and full text in the transcript

Keep the measured timing boundary visible

- 01** Batch 1 median: 0.881265 ms
- 02** Measured on GB10 after warmup
- 03** Excludes capture, flow waiting, preprocessing and transfers

A possible SmartNIC and NPU division



Follow the direction; each stage has a defined input and output.

Weights are not a portable computation graph

- 01** Model-only export: saved parameter tensors
- 02** Graph export: represent the computation for another runtime
- 03** Tested strict graph capture failed at a custom CUDA operator

A defensible hardware validation process

- 01** Check full-model output and gradient parity where relevant
- 02** Measure quantization accuracy on labeled data
- 03** Measure throughput, tail latency, buffering and drops

What remains missing

- 01** Matching paper accuracy and full pretraining
- 02** Verified live extraction and customer-network validation
- 03** Calibration, unknown attacks and deployed hardware backend

Teach the project to someone else

? What data went in, and why?

? What was executed and measured?

? What remains before customer deployment?

Your turn · 90 seconds

01 Explain the data choice, tested result, repository and remaining deployment work.

Speak or write your answer. The lesson resumes automatically.

Evidence: answers · walkthrough · results | Links and full text in the transcript

Your concise explanation

- 01** We built a checked execution and evidence layer
- 02** The original model trained on compatible native flows
- 03** Measured results and limits are both available